

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ_____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА _____СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Дисциплина: Парадигмы и конструкции языков программирования

Отчёт по лабораторной работе № 1

«Основные конструкции языка Python»

Выполнил: студент группы ИУ5-31Б _____ Синявский С.Д.

Проверил: _____ _____

2026 г.

1. Описание задания

Цель лабораторной работы: изучение основных конструкций языка Python.

Разработать программы для решения биквадратного уравнения $A \cdot x^4 + B \cdot x^2 + C = 0$.

1. Программы должны быть разработаны в виде консольного приложения на языке Python. Разработайте три программы на языке Python:

1) с применением процедурной парадигмы (использование функций, оператор def);

2) с применением объектно-ориентированной парадигмы (использование классов);

3) с применением функциональной парадигмы (сопоставление с образцом).

2. Каждая программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).

3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки. Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2.

4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

2. Текст программы

2.1. Файл biquadratic_proc.py (процедурная парадигма)

```
import sys
import math


def get_coef(index, prompt):
    if index < len(sys.argv):
        try:
            return float(sys.argv[index])
        except ValueError:
            pass
    while True:
        print(prompt)
        try:
            return float(input())
        except ValueError:
            pass


def get_discriminant(a, b, c):
    return b*b - 4*a*c


def get_roots(a, b, c):
    D = get_discriminant(a, b, c)
    square_roots = []
    if a == 0.0:
        if b != 0.0:
            square_roots.append(-c / b)
    elif D == 0.0:
        square_roots.append(-b / (2.0*a))
    elif D > 0.0:
        sqD = math.sqrt(D)
        square_roots.append((-b + sqD) / (2.0*a))
        square_roots.append((-b - sqD) / (2.0*a))
    result = []
    for t in square_roots:
        if t == 0.0:
            result.append(0.0)
        elif t > 0.0:
            result.append(-math.sqrt(t))
            result.append(math.sqrt(t))
    return sorted(result)


def main():
    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
    if a == 0.0 and b == 0.0 and c == 0.0:
        print('Корень - любое число')
        return
    if a == 0.0:
        print('A = 0, уравнение не биквадратное')
    else:
        print('Дискриминант: {}'.format(get_discriminant(a, b, c)))
    roots = get_roots(a, b, c)
    if len(roots) == 0:
        print('Нет действительных корней')
    else:
        print('Корни: {}'.format(', '.join(str(root) for root in roots)))
```

```
if __name__ == "__main__":
    main()
```

2.2. Файл biquadratic_oop.py (объектно-ориентированная парадигма)

```
import sys
import math


class BiquadraticEquation:

    def __init__(self):
        self.coef_A = 0.0
        self.coef_B = 0.0
        self.coef_C = 0.0
        self.discriminant = 0.0
        self.roots_list = []

    def get_coef(self, index, prompt):
        if index < len(sys.argv):
            try:
                return float(sys.argv[index])
            except ValueError:
                pass
        while True:
            print(prompt)
            try:
                return float(input())
            except ValueError:
                pass

    def get_coefs(self):
        self.coef_A = self.get_coef(1, 'Введите коэффициент A:')
        self.coef_B = self.get_coef(2, 'Введите коэффициент B:')
        self.coef_C = self.get_coef(3, 'Введите коэффициент C:')

    def calculate_roots(self):
        a = self.coef_A
        b = self.coef_B
        c = self.coef_C
        self.discriminant = b*b - 4*a*c
        D = self.discriminant
        square_roots = []
        if a == 0.0:
            if b != 0.0:
                square_roots.append(-c / b)
        elif D == 0.0:
            square_roots.append(-b / (2.0*a))
        elif D > 0.0:
            sqD = math.sqrt(D)
            square_roots.append((-b + sqD) / (2.0*a))
            square_roots.append((-b - sqD) / (2.0*a))
        for t in square_roots:
            if t == 0.0:
                self.roots_list.append(0.0)
            elif t > 0.0:
                self.roots_list.append(-math.sqrt(t))
                self.roots_list.append(math.sqrt(t))
        self.roots_list.sort()

    def print_roots(self):
        if self.coef_A == 0.0 and self.coef_B == 0.0 and self.coef_C == 0.0:
            print('Корень - любое число')
            return
        if self.coef_A == 0.0:
```

```
            print('A = 0, уравнение не биквадратное')
        else:
            print('Дискриминант: {}'.format(self.discriminant))
        if len(self.roots_list) == 0:
            print('Нет действительных корней')
        else:
            print('Корни: {}'.format(', '.join(str(root) for root in self.roots_list)))


def main():
    eq = BiquadraticEquation()
    eq.get_coefs()
    eq.calculate_roots()
    eq.print_roots()


if __name__ == "__main__":
    main()
```

2.3. Файл biquadratic_func.py (функциональная парадигма)

```
import sys
import math
from typing import NamedTuple


class SquareRootResult:
    NoRoots = NamedTuple("NoRoots", [])
    OneRoot = NamedTuple("OneRoot", [("root", float)])
    TwoRoots = NamedTuple("TwoRoots", [("root1", float), ("root2", float)])
    AnyRoot = NamedTuple("AnyRoot", [])


def parse_coef(coef_str):
    try:
        return float(coef_str)
    except ValueError:
        return None


def read_coef(prompt):
    print(prompt)
    match parse_coef(input()):
        case float(coef):
            return coef
        case _:
            return read_coef(prompt)


def get_coef(index, prompt):
    match [parse_coef(arg) for arg in sys.argv[index:index + 1]]:
        case [float(coef)]:
            return coef
        case _:
            return read_coef(prompt)


def get_discriminant(a, b, c):
    return b*b - 4*a*c


def get_square_roots(a, b, c):
    match (a, b, c):
        case (0, 0, 0):
            return SquareRootResult.AnyRoot()
        case (0, 0, _):
            return SquareRootResult.NoRoots()
        case (0, _, _):
            return SquareRootResult.OneRoot(-c / b)
    match get_discriminant(a, b, c):
        case 0:
            return SquareRootResult.OneRoot(-b / (2.0*a))
        case D if D > 0:
            sqD = math.sqrt(D)
            return SquareRootResult.TwoRoots((-b + sqD) / (2.0*a), (-b - sqD) / (2.0*a))
        case _:
            return SquareRootResult.NoRoots()


def roots_from_square(t):
    match t:
```

```
        case 0:
            return (0.0,)
        case t if t > 0:
            return (-math.sqrt(t), math.sqrt(t))
        case _:
            return ()


def get_roots(a, b, c):
    match get_square_roots(a, b, c):
        case SquareRootResult.AnyRoot():
            return None
        case SquareRootResult.NoRoots():
            return ()
        case SquareRootResult.OneRoot(t):
            return roots_from_square(t)
        case SquareRootResult.TwoRoots(t1, t2):
            return tuple(sorted(roots_from_square(t1) + roots_from_square(t2)))


def discriminant_text(a, b, c):
    match (a, b, c):
        case (0, 0, 0):
            return None
        case (0, _, _):
            return 'A = 0, уравнение не биквадратное'
        case _:
            return 'Дискриминант: {}'.format(get_discriminant(a, b, c))


def roots_text(roots):
    match roots:
        case None:
            return 'Корень - любое число'
        case ():
            return 'Нет действительных корней'
        case _:
            return 'Корни: {}'.format(', '.join(str(root) for root in roots))


def main():
    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
    match discriminant_text(a, b, c):
        case str(text):
            print(text)
    print(roots_text(get_roots(a, b, c)))


if __name__ == "__main__":
    main()
```

3. Экранные формы с примерами выполнения программы

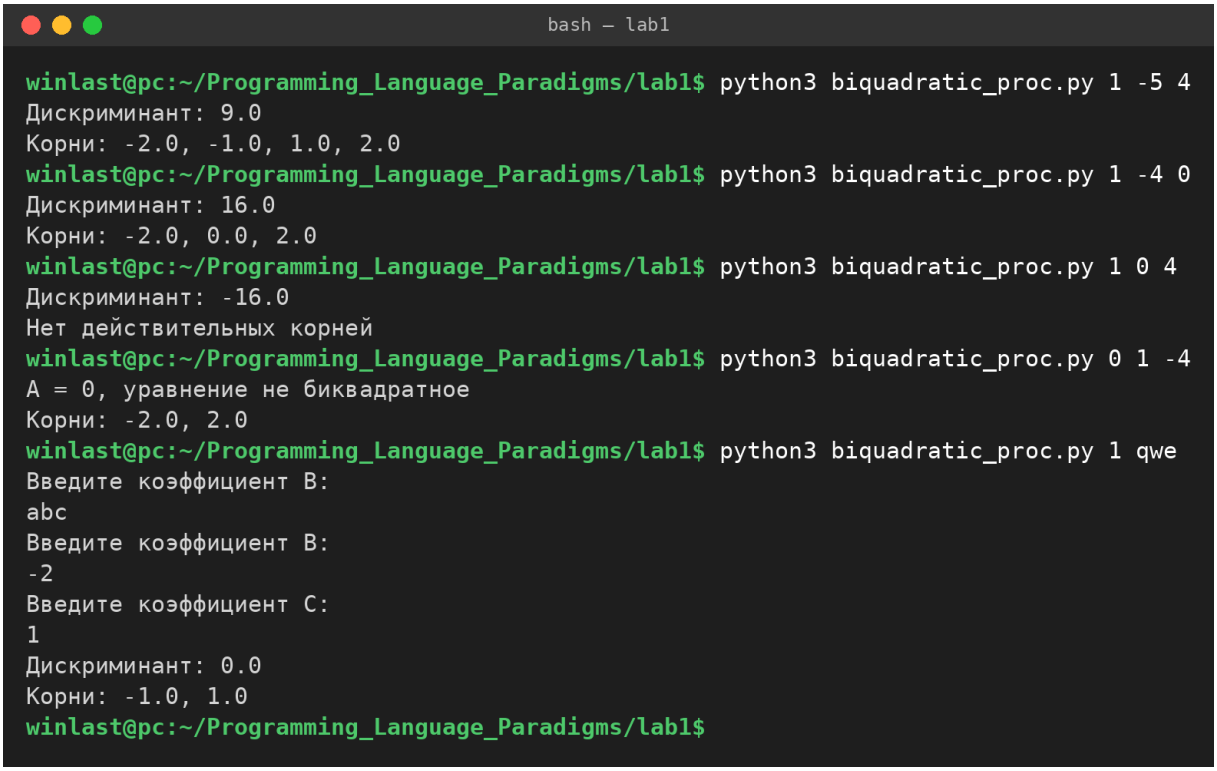


Рисунок 1 — Выполнение программы biquadratic_proc.py

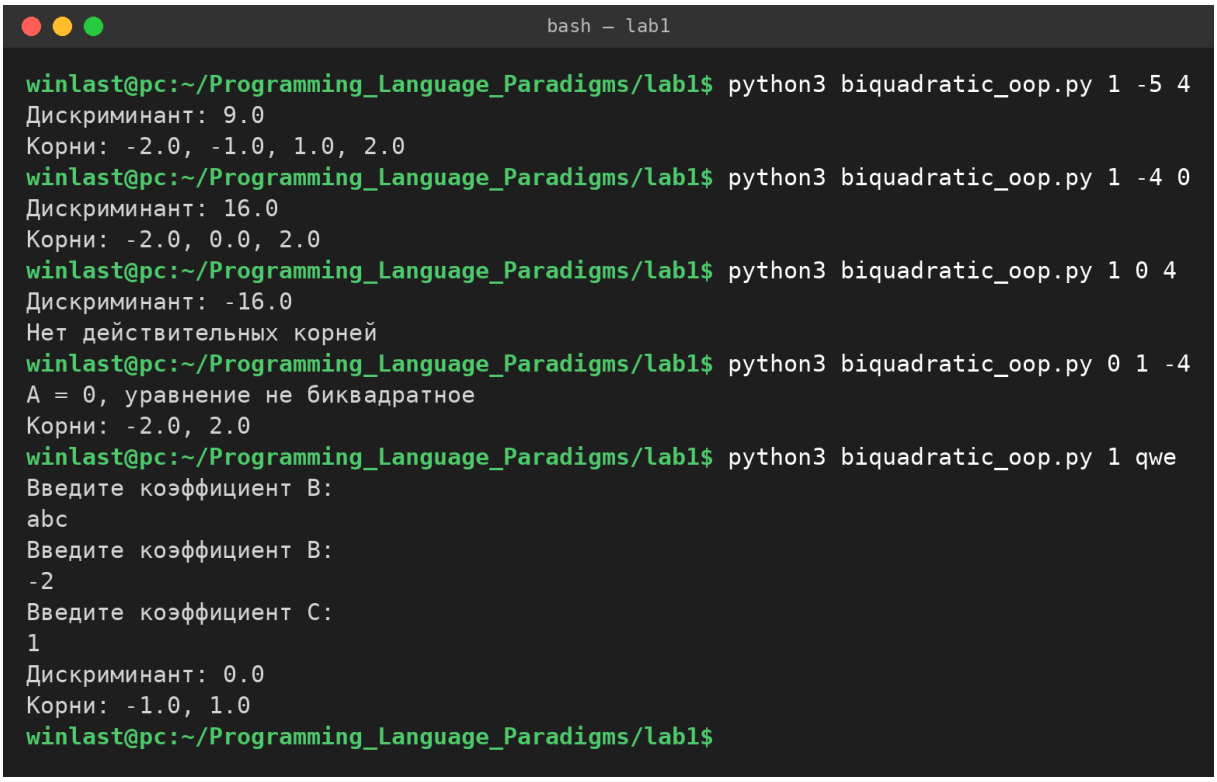


Рисунок 2 — Выполнение программы biquadratic_oop.py

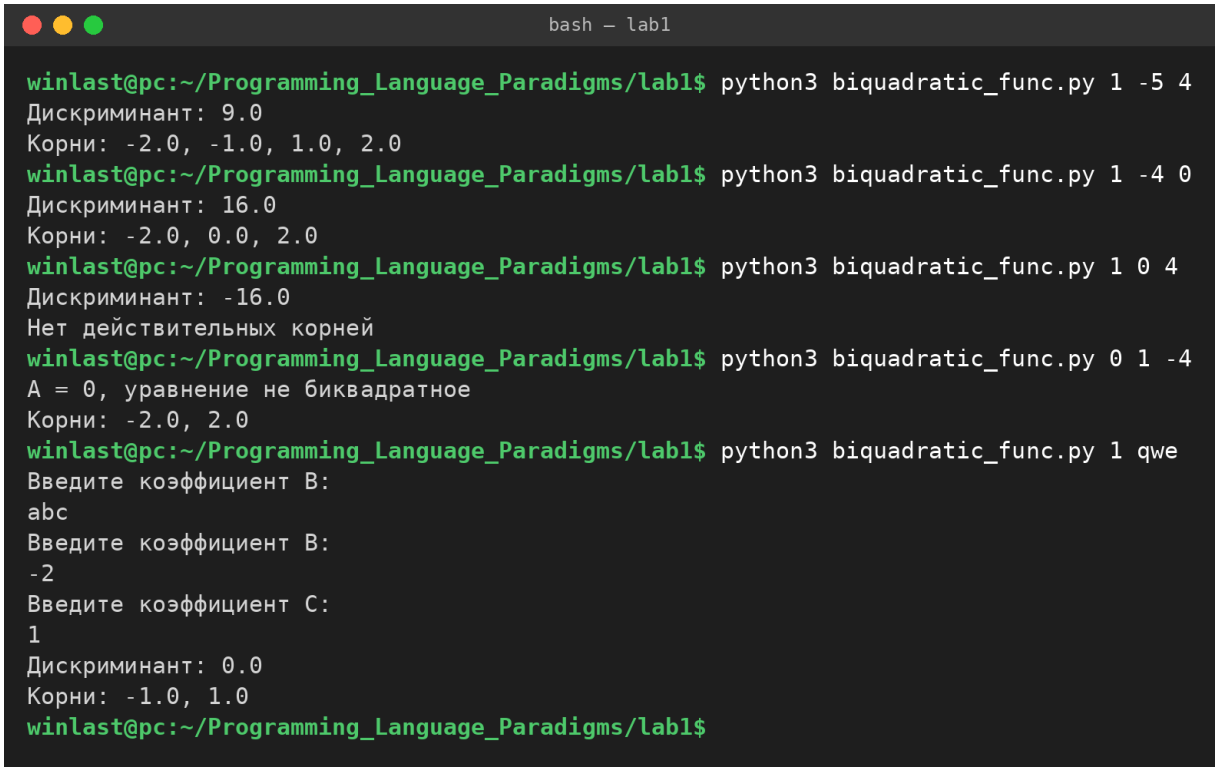


Рисунок 3 — Выполнение программы biquadratic_func.py